

# **COURIER MANAGEMENT SYSTEM**

PROJECT REPORT

**Submitted By**

H. Aasif

B. Brain Rukshan

S. Prathap

M. Parthasarathy

Department of Artificial Intelligence

E.S. Arts and Science College (Co-Ed), Villupuram

# **TABLE OF CONTENTS**

## **ABSTRACT**

## **1. INTRODUCTION**

### **1.1 BACKGROUND OF THE STUDY**

### **1.2 CHALLENGES IN MANUAL COURIER MANAGEMENT**

### **1.3 OBJECTIVES AND SCOPE OF THE PROJECT**

## **2. LITERATURE SURVEY**

### **2.1 OVERVIEW OF EXISTING COURIER MANAGEMENT METHODS**

### **2.2 MANUAL Vs AUTOMATED COURIER SYSTEMS**

### **2.3 ALGORITHMS AND TECHNIQUES IN COURIER MANAGEMENT**

### **2.4 RELATED WORKS ON DELIVERY MANAGEMENT SYSTEMS**

## **3. SYSTEM ANALYSIS AND DESIGN**

### **3.1 REQUIREMENT ANALYSIS**

### **3.2 SYSTEM ARCHITECTURE**

### **3.3 DATA FLOW DIAGRAMS (DFD)**

### **3.4 OUTPUT SCREENS**

### **3.5 ENTITY RELATIONSHIP DIAGRAM (ERD)**

### **3.6 DESIGN CONSTRAINTS AND ASSUMPTIONS**

## **4. IMPLEMENTATION OF THE SYSTEM**

### **4.1 TECHNOLOGY STACK OVERVIEW**

### **4.2 DATABASE DESIGN**

### **4.3 MODULE DESCRIPTIONS**

### **4.4 USER INTERFACE DESIGN**

### **4.5 SYSTEM INTEGRATION**

## **5. TESTING AND EVALUATION**

### **5.1 TESTING STRATEGY & TYPES OF TESTING**

### **5.2 PERFORMANCE EVALUATION**

### **5.3 ERROR HANDLING & DEBUGGING**

## **6. CONCLUSION AND FUTURE WORK**

6.1 SUMMARY OF CONTRIBUTIONS

6.2 ACHIEVEMENTS OF THE SYSTEM

6.3 RECOMMENDATIONS FOR FUTURE ENHANCEMENTS

REFERENCES

## ABSTRACT

Courier delivery is an essential service for individuals, businesses, educational institutions, and online commerce. Traditional courier operations often depend on paper records, phone calls, spreadsheets, and manual status updates. These methods can cause delays, misplaced records, duplicate entries, difficulty in tracking parcels, and poor communication between customers, staff, and delivery personnel. To address these challenges, this project proposes a **Courier Management System** that centralizes parcel booking, customer information, shipment tracking, delivery assignment, status management, and report generation.

The system is designed to manage the complete courier lifecycle from parcel registration to final delivery. It stores sender and receiver details, parcel information, pickup and delivery addresses, tracking numbers, delivery status, assigned delivery personnel, and timestamps. Administrators can register shipments, update parcel status, assign delivery staff, search tracking information, and generate reports. Customers can use a tracking number to view the current shipment status.

A web-based architecture using HTML, CSS, JavaScript, a Python backend such as Flask or Django, and a relational database such as MySQL, PostgreSQL, or SQLite can be used for implementation. The modular design allows future integration of notifications, route optimization, online payments, barcode scanning, and mobile applications.

# **CHAPTER 1**

## **INTRODUCTION**

Courier services play an important role in modern communication and commerce by transporting documents, parcels, products, and other items between locations. As the number of shipments increases, courier organizations must manage large amounts of customer, parcel, payment, delivery, and tracking information. Manual record keeping becomes difficult when many shipments are processed simultaneously.

A Courier Management System provides a centralized platform for recording shipments and monitoring their progress. Instead of depending on separate paper registers or spreadsheets, authorized staff can enter and retrieve information through a web interface. The system can maintain a unique tracking number for each shipment and record events such as booking, pickup, dispatch, arrival at a hub, out-for-delivery, and delivered.

### **1.1 BACKGROUND OF THE STUDY**

Courier operations involve several connected activities including customer registration, shipment booking, parcel classification, payment recording, pickup, hub processing, delivery assignment, tracking, and final delivery confirmation. When these activities are handled manually, information may be entered multiple times and status changes may not be communicated quickly.

The growth of online shopping has increased expectations for accurate shipment tracking and timely delivery. Customers generally want to know where a parcel is and whether it is expected to arrive. A centralized courier system can maintain a consistent shipment history and make tracking information available to authorized users.

### **1.2 CHALLENGES IN MANUAL COURIER MANAGEMENT**

- Time-consuming registration of shipments and customer information.
- Duplicate or incorrect entries in paper registers and spreadsheets.
- Difficulty in locating a parcel when the tracking number or status is needed.
- Delayed status updates between booking staff, hubs, and delivery personnel.
- Difficulty assigning delivery staff and monitoring pending deliveries.
- Limited ability to generate accurate daily, weekly, or monthly reports.
- Risk of damaged, lost, or incomplete records when information is stored manually.

### **1.3 OBJECTIVES AND SCOPE OF THE PROJECT**

The main objective of the Courier Management System is to automate and organize the major activities involved in parcel booking and delivery tracking. The system is intended to reduce repetitive manual work, improve information accuracy, and provide quick access to shipment status.

- Manage sender and receiver information.

- Register parcels and generate unique tracking numbers.
- Record parcel type, weight, dimensions, charges, and delivery address.
- Assign shipments to delivery personnel and update delivery status.
- Provide tracking search based on tracking number.
- Maintain shipment history and timestamps.
- Generate operational reports for administrators.
- Provide authentication and role-based access for authorized users.

## **CHAPTER 2**

### **LITERATURE SURVEY**

Existing courier operations range from manual registers and spreadsheets to integrated logistics platforms. The level of automation depends on organizational size, shipment volume, budget, and technical infrastructure. Common delivery-management designs emphasize centralized data handling, tracking, automation, route planning, and reporting.

#### **2.1 OVERVIEW OF EXISTING COURIER MANAGEMENT METHODS**

Manual courier management uses physical registers or spreadsheets to record customer and shipment details. It is easy to start but becomes difficult to maintain as shipment volume increases. Staff must search records manually and update the same information in several places.

Semi-automated methods use spreadsheets or simple database applications. These tools improve organization but may still require manual status updates and staff intervention for assignment, tracking, and reporting.

Modern courier management platforms use centralized databases and web or mobile interfaces. They can provide tracking numbers, delivery status updates, staff assignment, notifications, and analytical reports.

#### **2.2 MANUAL VS AUTOMATED COURIER SYSTEMS**

Manual systems require staff to enter, search, and update information repeatedly. Automated systems centralize shipment information and use software rules to validate data, generate tracking identifiers, update status, and produce reports.

- Manual: paper/spreadsheet records; Automated: centralized database.
- Manual: status updates depend on staff; Automated: status history can be recorded through system actions.
- Manual: tracking requires searching registers; Automated: tracking number provides quick lookup.
- Manual: reports require manual preparation; Automated: reports can be generated from stored data.
- Manual: higher risk of duplicate or inconsistent records; Automated: validation and unique identifiers reduce these risks.

#### **2.3 ALGORITHMS AND TECHNIQUES IN COURIER MANAGEMENT**

Courier systems can use rule-based processing to validate shipment status transitions and assign delivery personnel. Search and indexing techniques make tracking-number lookup fast. Queue-based processing can organize parcels waiting for pickup, hub transfer, or delivery.

Route optimization algorithms can be used when geographic information is available. Heuristic methods can create practical delivery sequences quickly, while advanced optimization can consider distance, delivery windows, vehicle capacity, and traffic information. Barcode or QR-code scanning can reduce manual entry during parcel movement.

## **2.4 RELATED WORKS ON DELIVERY MANAGEMENT SYSTEMS**

Delivery management platforms commonly combine shipment booking, tracking, delivery assignment, notifications, and reporting. Enterprise logistics systems may provide extensive functionality but can be complex for smaller organizations. A focused Courier Management System can provide the core functions required by a local courier office while leaving advanced integrations for future enhancement.

## CHAPTER 3

### SYSTEM ANALYSIS AND DESIGN

#### 3.1 REQUIREMENT ANALYSIS

Requirement analysis identifies the functions and quality attributes required by the Courier Management System. The primary users are administrators or courier staff, delivery personnel, and customers who need shipment tracking information.

##### Functional Requirements

- Allow authorized staff to create and update customer records.
- Register new courier shipments and generate a unique tracking number.
- Store sender, receiver, parcel, address, charge, and booking information.
- Assign shipments to delivery personnel.
- Support statuses such as Booked, Picked Up, In Transit, At Hub, Out for Delivery, Delivered, and Cancelled.
- Search a shipment using its tracking number.
- Maintain shipment history for auditing and reference.
- Generate reports for shipments, deliveries, pending parcels, and revenue.

##### Non-Functional Requirements

- Simple and usable interface on common browsers.
- Authentication and authorization for data protection.
- Fast search and status retrieval.
- Scalable database for increasing customers and shipments.
- Reliable error handling and validation.

#### 3.2 SYSTEM ARCHITECTURE

The proposed system follows a client-server architecture. Users interact with a web frontend through a browser. The frontend sends requests to the backend, where business rules are applied. The backend communicates with a relational database that stores customers, shipments, delivery staff, payments, and tracking history.

|                                             |
|---------------------------------------------|
| Customer / Staff / Delivery Agent           |
| Web Interface (HTML + CSS + JavaScript)     |
| Backend Application (Python Flask / Django) |

Database (MySQL / PostgreSQL / SQLite)

Figure 3.2: Courier Management System Architecture

### 3.3 DATA FLOW DIAGRAMS (DFD)

At Level 0, customers and courier staff interact with the Courier Management System. At Level 1, the system can be divided into Customer Management, Shipment Booking, Delivery Assignment, Tracking, and Report Generation processes.

| External Entity | Process             | Main Data                        |
|-----------------|---------------------|----------------------------------|
| Customer        | Shipment Booking    | Sender, receiver, parcel details |
| Staff           | Shipment Processing | Tracking number, charges, status |
| Delivery Agent  | Delivery Update     | Pickup/delivery confirmation     |
| Admin           | Reporting           | Shipment and delivery records    |

Table 3.3: Major Courier Data Flows

### 3.4 OUTPUT SCREENS

The proposed interface contains simple screens for authentication, dashboard access, customer management, shipment booking, shipment tracking, delivery assignment, and reporting.

**Figure 3.4:** Login Page

**Figure 3.4:** Dashboard

**Figure 3.4:** Customer Management

**Figure 3.4:** New Shipment / Booking

**Figure 3.4:** Shipment Tracking

**Figure 3.4:** Delivery Agent Management

**Figure 3.4:** Shipment Status Update

**Figure 3.4:** Reports

### 3.5 ENTITY RELATIONSHIP DIAGRAM (ERD)

The database model contains entities such as Customer, Shipment, Delivery Agent, Payment, and Tracking History. A customer may create multiple shipments. Each shipment has one tracking number and can have multiple tracking-history records. A delivery agent may be assigned to multiple shipments over time.

| CUSTOMER        | SHIPMENT        | DELIVERY AGENT |
|-----------------|-----------------|----------------|
| CustomerID (PK) | ShipmentID (PK) | AgentID (PK)   |
| Name            | TrackingNumber  | Name           |
| Phone           | SenderID (FK)   | Phone          |
| Address         | ReceiverID (FK) | Status         |
|                 | AgentID (FK)    |                |

Figure 3.5: Simplified Courier ERD

### **3.6 DESIGN CONSTRAINTS AND ASSUMPTIONS**

- The system depends on accurate customer and shipment data.
- Internet or local network access is required for web deployment.
- Delivery status depends on timely updates by staff or delivery personnel.
- Advanced live GPS tracking is outside the basic scope.
- Payment gateway integration is optional for the basic implementation.
- Users are assumed to have basic computer or smartphone skills.

# CHAPTER 4

## IMPLEMENTATION OF THE SYSTEM

### 4.1 TECHNOLOGY STACK OVERVIEW

The Courier Management System can be implemented using standard web technologies. HTML structures the interface, CSS provides responsive styling, and JavaScript handles client-side interaction and validation. A Python backend using Flask or Django can implement application logic and APIs. MySQL, PostgreSQL, or SQLite can store structured courier data. The original report uses the same general frontend/backend/database pattern and identifies HTML, CSS, JavaScript, Python frameworks, and relational databases as suitable technologies.

Development tools such as Visual Studio Code and Git can support coding and version control. Reporting libraries can generate PDF or spreadsheet reports when required.

### 4.2 DATABASE DESIGN

A relational database is suitable because courier records contain clear relationships between customers, shipments, delivery agents, payments, and tracking events.

| Table           | Main Fields                                                                                 |
|-----------------|---------------------------------------------------------------------------------------------|
| Customers       | CustomerID, Name, Phone, Email, Address                                                     |
| Shipments       | ShipmentID, TrackingNumber, SenderID, ReceiverID, BookingDate, Weight, Type, Charge, Status |
| DeliveryAgents  | AgentID, Name, Phone, VehicleNumber, Availability                                           |
| TrackingHistory | HistoryID, ShipmentID, Status, Location, UpdatedAt                                          |
| Payments        | PaymentID, ShipmentID, Amount, Method, PaymentStatus                                        |
| Users           | UserID, Username, PasswordHash, Role                                                        |

Table 4.2: Courier Database Tables

### 4.3 MODULE DESCRIPTIONS

#### Admin Module

Controls users, roles, system settings, and overall courier operations.

#### Customer Module

Stores sender and receiver information and allows staff to update customer details.

#### Shipment Booking Module

Registers parcels, calculates basic charges, creates tracking numbers, and records booking information.

#### Tracking Module

Displays current shipment status and tracking history using the tracking number.

#### Delivery Management Module

Assigns shipments to delivery agents and records pickup and delivery updates.

## **Payment Module**

Records shipment charges and payment status.

## **Report Generation Module**

Produces shipment, delivery, pending parcel, and revenue reports.

## **4.4 USER INTERFACE DESIGN**

The interface is designed to be simple, responsive, and consistent. A dashboard can display total shipments, pending deliveries, parcels in transit, and completed deliveries. Forms should use clear labels and validation messages. Search and filter controls can help staff locate shipments quickly.

- Responsive layout for desktop, laptop, and mobile screens.
- Clear navigation between customers, shipments, tracking, agents, and reports.
- Validation for phone numbers, required addresses, parcel weight, and shipment status.
- Readable tables with tracking numbers and current status.
- Confirmation messages after booking, assignment, or status updates.

## **4.5 SYSTEM INTEGRATION**

System integration connects the frontend, backend, and database through application APIs. When a staff member books a parcel, the frontend sends shipment data to the backend. The backend validates the input, stores the shipment, creates a tracking number, and returns the result to the interface. Later status updates are stored in tracking history and can be displayed through the tracking screen.

Authentication and authorization should be integrated across the system so that administrative functions are restricted to authorized users. Import and export features can be added for CSV and PDF reports.

## **CHAPTER 5**

### **TESTING AND EVALUATION**

#### **5.1 TESTING STRATEGY & TYPES OF TESTING**

Testing verifies that each module works independently and that the integrated system performs the expected courier workflow. Testing should begin with small functions and continue through complete shipment booking and delivery scenarios.

##### **Unit Testing**

Tests functions such as tracking-number generation, charge calculation, validation, and status changes.

##### **Integration Testing**

Checks communication between shipment booking, database storage, tracking, and delivery assignment.

##### **System Testing**

Tests the complete workflow from login and booking to delivery completion and reporting.

##### **User Acceptance Testing (UAT)**

Allows intended users to verify whether the system is understandable and meets operational requirements.

#### **5.2 PERFORMANCE EVALUATION**

Performance evaluation can measure response time for tracking searches, shipment registration, dashboard loading, report generation, and database operations. The system should remain responsive as the number of shipment records increases. Database indexing on tracking numbers can improve search performance.

- Tracking search should return the shipment record quickly.
- Shipment booking should validate and save data without unnecessary delay.
- Reports should be generated within a practical time for the selected date range.
- The database should support increasing shipment and customer records.

#### **5.3 ERROR HANDLING AND DEBUGGING**

Error handling is important because courier records directly affect delivery operations. Input validation should detect missing addresses, invalid contact information, duplicate tracking numbers, and invalid status transitions. Backend exceptions should be logged while the user receives a clear message rather than a technical error.

- Frontend validation for required fields and basic formats.
- Backend validation for business rules and database constraints.
- Unique constraint on tracking numbers.

- Safe handling of failed database operations.
- Logging of errors with timestamps and relevant module information.
- User-friendly recovery messages for common problems.

## **CHAPTER 6**

### **CONCLUSION AND FUTURE WORK**

#### **6.1 SUMMARY OF CONTRIBUTIONS**

The Courier Management System provides a structured digital approach to courier operations. It brings customer management, shipment booking, tracking, delivery assignment, payment recording, and reporting into one platform. A centralized database reduces repeated manual record keeping and makes shipment information easier to retrieve.

- Centralized management of customer and shipment records.
- Unique tracking numbers for shipment identification.
- Status history for monitoring parcel movement.
- Delivery-agent assignment and delivery updates.
- Report generation for operational monitoring.
- Role-based access for improved data protection.

#### **6.2 ACHIEVEMENTS OF THE SYSTEM**

The proposed system achieves the core goal of organizing courier information and simplifying routine delivery management. It provides a foundation for faster shipment lookup, clearer status tracking, reduced duplicate records, and better administrative visibility. The modular structure also allows individual components to be improved without redesigning the entire system.

#### **6.3 RECOMMENDATIONS FOR FUTURE ENHANCEMENTS**

- Real-time GPS tracking for delivery vehicles and agents.
- SMS, email, or app notifications for shipment status changes.
- Barcode and QR-code scanning at booking and hub checkpoints.
- Online payment gateway integration.
- Route optimization using maps and delivery locations.
- Mobile application for delivery agents and customers.
- Analytics dashboard for delivery time, pending shipments, and operational trends.
- Integration with e-commerce platforms and external logistics APIs.
- Two-factor authentication and stronger security controls.

## REFERENCES

- [1] Database management principles for relational data, transactions, keys, and indexing.
- [2] Python Flask / Django documentation and web application development practices.
- [3] HTML, CSS, and JavaScript web development documentation.
- [4] Software engineering principles for requirements analysis, system design, testing, and maintenance.
- [5] General courier and logistics management system design concepts.

The uploaded report was used as the structural basis for this conversion; its original organization contains the abstract, introduction, literature survey, system analysis and design, implementation, testing, conclusion, and references.